



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

تقرير مشروع البرمجة المتوازية 2026

تنفيذ الطلاب

سليمان مظهر يوسف

نايا خليل صالحه

علاء الدين الأمين

نور عامر شاهين

زين سمير أحمد

بإشراف المهندس :

حذيفة العقيل

١- المتطلب الأول : (Concurrent Access & Data Integrity)

بداية للطلب الأول احتجنا لاستخدام لاراغون بدلا من سيرفر لارافيل العادي لأننا نحتاج اختبار ومعالجة طلبات متزامنة (**concurrent requests**) وقياس سلوك النظام تحت حمل حقيقي. خادم `php artisan serve` مخصص للتطوير البسيط ويعمل في عملية واحدة ولا يعكس سلوك الخادم الحقيقي عند استقبال طلبات متزامنة أو عند التعامل مع اتصالات متعددة، بينما Laragon يوفر بيئة أقرب للإنتاج (Apache/Nginx + PHP-FPM) ويسمح بتشغيل عدة عمليات متزامنة، ضبط المنافذ، وإدارة قواعد البيانات والخدمات بسهولة.

لاختبار ال `race conditation` طبقنا ذلك على عملية الاوردر حيث ان المستخدم هنا يحتاج لاختذ المنتج من المستودع بالداتا بيز و لكن في حال قام اكثر من مستخدم في نفس الوقت بالقيام بنفس العملية قد تقع ب `race conditation` و هذا ما واجهناه بالكود التالي للقيام بطلب معين :

```
return response()->json([ 'error' => 'Some cart items are invalid '], 400);
}

try {
    $order = DB::transaction(function () use ($user, $cartItems, $cart, $CartItem) {

        $productIds = $cartItems->pluck('product_id');
        $products = Product::whereIn('id', $productIds)
            ->get()
            ->keyBy('id');

        foreach ($cartItems as $item) {
            $product = $products->get($item->product_id);

            if (!$product) {
                throw new \Exception("Product not found for id {$item->product_id}");
            }

            if ($product->stock < $item->quantity) {
                throw new \App\Exceptions\InsufficientStockException(
                    "Requested quantity exceeds available stock",
                    $product->name
                );
            }
        }

        $totalPrice = $cartItems->sum(fn($i) => $i->price * $i->quantity);

        $order = Order::create([
            'user id' => $user->id,
```

لاختبار قيام اكثر من مستخدم بالطلب استخدمنا ال k6 حيث k6 يتيح تشغيل آلاف الطلبات متزامنة بطريقة قابلة للبرمجة، ما يكشف عن Race Conditions و مشاكل قفل قواعد البيانات .

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  vus: 20,
  iterations: 20,
  // duration: '30s',
};

export default function () {

  const payload = JSON.stringify({
    items: [40]
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer eyJ0eXAiOiJKV1QiLCJ...
    },
  };

  let res = http.post(
    'http://ppp.test/api/orders/create',
    payload,
    params
  );

  // check(res, {
  //   'status exists': (r) => r.status > 0,
  // });
  console.log(res.status);
  console.log(res.body);
}
```

سكربت k6 بسيط (20 VUs × 20 iterations) لإرسال طلبات POST إلى /api/orders/create بهدف محاكاة حمل متزامن واكتشاف مشكلات تنافسية أو أخطاء في المصادقة. السكربت يرسل حمولة JSON مع توكن مصادقة ويسجل حالة الاستجابة ومحتواها. و ليكن لدينا منتج بالداتا بيز :

94	blanditiis	Suscipit sapiente dolore nam sunt. Id cumque qui eos q...	130.0	21	2026-05-16 15:55:46	20
----	------------	---	-------	----	---------------------	----

و طبقنا التست بحيث كل مستخدم يأخذ قطعتين من المنتج تكون النتيجة بعد تطبيق الامر k6 run race_test.js :

```
INFO[0007] {"message":"Order created successfully","order_id":828,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":829,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":830,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":831,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":832,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":833,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":834,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":835,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":836,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":837,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":839,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":840,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":838,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":841,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":842,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":844,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":843,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":846,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":845,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":847,"total_price":520} source=console

data_received.....: 7.5 kB 678 B/s
data_sent.....: 10 kB 902 B/s
http_req_blocked.....: avg=8.85ms min=1.83ms med=8.68ms max=21ms p(90)=12.08ms p(95)=12.78ms
http_req_connecting.....: avg=7.53ms min=1.64ms med=7.67ms max=19.75ms p(90)=10.21ms p(95)=10.71ms
http_req_duration.....: avg=11.57s min=9.64s med=10.46s max=33.86s p(90)=10.99s p(95)=12.17s
{ expected_response:true }...: avg=11.57s min=9.64s med=10.46s max=33.86s p(90)=10.99s p(95)=12.17s
http_req_failed.....: 0.00% 0 out of 20
http_req_receiving.....: avg=12.37ms min=194.92µs med=10.02ms max=31.14ms p(90)=23.4ms p(95)=28.71ms
http_req_sending.....: avg=8.66ms min=122.47µs med=3.01ms max=21.18ms p(90)=20.9ms p(95)=21.03ms
http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
http_req_waiting.....: avg=11.54s min=9.59s med=10.43s max=33.83s p(90)=10.98s p(95)=12.16s
http_reqs.....: 20 1.806986/s
iteration_duration.....: avg=9.74s min=7.06s med=9.88s max=10.53s p(90)=10.44s p(95)=10.47s
iterations.....: 20 1.806986/s
vus.....: 8 min=8 max=20
vus_max.....: 20 min=20 max=20

running (00m11.1s), 00/20 VUs, 20 complete and 0 interrupted iterations
default ✓ [=====] 20 VUs 00m10.5s/10m0s 20/20 shared iters
```

مع ان النتيجة المتوقعة يجب ان تكون : فقط 10 مستخدمين يستطيعون القيام بالعملية بنجاح و لكن هنا نلاحظ ان الجميع قامو بالعملية و كيف أصبح المنتج بالداتا بيز ؟

94	blanditiis	Suscipit sapiente dolore nam sunt. Id cumque qui eos q...	130.0	-19	2026-05-16 15:55:46	2026
----	------------	---	-------	-----	---------------------	------

نلاحظ قيمة سالبة و هذا شيء غير صحيح مع ان الشرط :

```
if ($product->stock < $item->quantity) {
    throw new \App\Exceptions\InsufficientStockException(
        "Requested quantity exceeds available stock",
        $product->name
    );
}
```

كان من المفترض ان يمنع بقية المستخدمين من الطلب لكن ما حصل ان العمليات المتوازية جعلت جميع المستخدمين قادرين على القيام بطلب المنتج بنفس الوقت .

لاصلاح ذلك قمنا بالتالي :

```
try {
    $order = DB::transaction(function () use ($user, $cartItems, $cart, $cartItemIds) {

        $productIds = $cartItems->pluck('product_id');
        $products = Product::whereIn('id', $productIds)
            ->lockForUpdate()
            ->get()
            ->keyBy('id');

        foreach ($cartItems as $item) {
            $product = $products->get($item->product_id);

            if (!$product) {
                throw new \Exception("Product not found for id {$item->product_id}");
            }

            if ($product->stock < $item->quantity) {
                throw new \App\Exceptions\InsufficientStockException(
                    "Requested quantity exceeds available stock",
                    $product->name
                );
            }
        }

        $totalPrice = $cartItems->sum(fn($i) => $i->price * $i->quantity);

        $order = Order::create([
            'user_id' => $user->id,
            'total_price' => $totalPrice,
        ]);

        foreach ($cartItems as $item) {
            $product = $products->get($item->product_id);

            // sleep(2);
            $product->decrement('stock', $item->quantity);

            OrderItem::create([
                'order_id' => $order->id,
                'product_id' => $item->product_id,
            ]);
        }
    });
}
```

أي قمنا بوضع قفل على المنتج كي نمنع ال Race Condition فنلاحظ بعد تكرار التجربة على منتج اخر :

2- المتطلب الثاني : (Resource Management & Capacity Control):

في البداية، كانت معمارية النظام تعتمد على المعالجة المتزامنة التقليدية (Synchronous Execution) ، حيث كان منطق الدفع بكامله يقع مباشرة داخل الدالة `createPaymentIntent` في الـ Controller دون الاستعانة بوظائف خلفية مؤجلة.

- الآثار السلبية لهذه المعمارية تحت الضغط:

- زمن استجابة مرتفع (High Latency) كان على المستخدم انتظار انتهاء عملية الدفع كاملة (والتي تتضمن الاتصال ببوابة الدفع الخارجي، التحقق من المخزن، وتحديث الجداول الحساسة) قبل استقبال رد الـ HTTP.
- اختناق السيرفر (Resource Exhaustion) مع تدفق الطلبات المتزامنة بكثافة، احتجز كل طلب خيط معالجة (Thread) كامل داخل الأباتشي لفترة طويلة. هذا الاحتجاز أدى سريعاً إلى استهلاك كامل الطاقة الاستيعابية للسيرفر، مما تسبب في انهيار العمليات (Crashes) وظهور أخطاء خادم داخلية (Status Code 500) قبل أن يتمكن إطار العمل حتى من تسجيل الخطأ.

- استراتيجية التحسين وتوزيع الموارد عبر Laravel Horizon :

- قمنا باستخدام Laravel Horizon كلوحة تحكم ونظام إدارة متقدم للطوابير تسمح لنا بتهيئة العمال (Workers) وتوزيع العمليات المتوازية (Processes) بدقة وعزل الموارد (Resource Isolation).
- لمعالجة هذا الاختناق، قمنا بنقل المهام الثقيلة (مثل معالجة الدفع وإصدار الفواتير) خارج المسار الرئيسي للطلب وتحويلها إلى جوبات خلفية غير متزامنة و (Asynchronous Jobs) و سيتم شرح أهمية و كيفية تطبيق هذه العملية بشكل مفصل في الطلب الثالث .
- شكل التابع بعد منطق الدفع ل job منفصل :

```

public function createPaymentIntent(Request $request)
{
    $validator = Validator::make($request->all(), [
        'order_id' => 'required|exists:orders,id',
        'scenario' => 'nullable|string'
    ]);

    if ($validator->fails()) {
        return response()->json([
            'error' => $validator->errors()
        ], 422);
    }

    $order = Order::where('user_id', Auth::id())
        ->where('status', 'pending')
        ->with('orderItems')
        ->find($request->order_id);

    if (!$order) {
        return response()->json([
            'message' => 'Order not found or not payable'
        ], 404);
    }

    if ($order->status == 'paid') {
        return response()->json([
            'message' => 'Order already paid'
        ], 400);
    }

    ProcessOrder::dispatch(
        $order,
        Auth::id(),
        $request->scenario
    )->onQueue('payments');

    return response()->json([
        'message' => 'Payment is being processed in background',
    ], 202);
}

```

أما بالنسبة لإعدادات ال horizon فتركناها بالبداية كما هي بالصورة :


```
'defaults' => [  
  'supervisor-default' => [  
    'connection' => 'redis',  
    'queue' => ['default'],  
    'balance' => 'auto',  
    'autoScalingStrategy' => 'time',  
    'maxProcesses' => 10,  
    'memory' => 128,  
    'tries' => 1,  
    'timeout' => 60,  
  ],  
,  
  
  'environments' => [  
    'production' => [  
      'supervisor-default' => [  
        'maxProcesses' => 20,  
      ],  
    ],  
  
    'local' => [  
      'supervisor-default' => [  
        'maxProcesses' => 3,  
      ],  
    ],  
  ],  
,
```

و قمنا بتطبيق تجربة باستخدام ال k6 بالشكل التالي :

[illegible]

حيث أنه تم انشاء 48 بالمئة من الطلبات اما بقية الطلبات فكما تحدثنا بالطلب الأول لم تستطع الوصول قبل غيرها للمستودع بالتالى لم تستطع اخذ القطع من المنتج أما عمليات الدفع فتمت كلها .

أداة **htop**: تم استخدام هذا الأمر التفاعلي داخل التيرمينال لمراقبة استهلاك المعالج (CPU) والذاكرة (Memory) لكل عملية (Process) بشكل لحظي أثناء فحص الضغط، لتحديد المكون الذي يشكل عبئاً على النظام.

عند تشغيل الامر http نلاحظ التالي :

Main		I/O																			
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command										
9103	mysql	20	0	2823M	467M	38692	S	0.7	12.3	0:05.60	/usr/sbin/mysqld										
9106	mysql	20	0	2823M	467M	38692	S	0.8	12.3	0:05.03	/usr/sbin/mysqld										
9724	nour	20	0	202M	70536	26004	S	0.0	1.8	0:17.33	/usr/bin/php8.3 artisan horizon:work	redis	--name=default	--superv							
9725	nour	20	0	202M	70536	26008	S	0.0	1.8	0:14.43	/usr/bin/php8.3 artisan horizon:work	redis	--name=default	--superv							
9726	nour	20	0	202M	70628	26124	S	0.7	1.8	0:15.18	/usr/bin/php8.3 artisan horizon:work	redis	--name=default	--superv							
11001	root	20	0	3188	1124	980	S	0.0	0.0	0:00.00	/init										
11003	root	20	0	3188	1256	1108	S	0.0	0.0	0:00.50	/init										
11006	nour	20	0	6068	5388	3644	S	0.0	0.1	0:00.71	-bash										
13137	mysql	20	0	2823M	467M	38692	S	0.0	12.3	0:02.68	/usr/sbin/mysqld										
13437	mysql	20	0	2823M	467M	38692	S	0.0	12.3	0:00.80	/usr/sbin/mysqld										
13440	mysql	20	0	2823M	467M	38692	S	0.0	12.3	0:01.07	/usr/sbin/mysqld										

نلاحظ ان :

•الازدحام في طابور واحد :يظهر الحقل (Queue: default) أن هذه المهمة الحساسة تم رميها في الطابور الافتراضي المشترك، مما يعني أنها كانت تتزاحم مع كافة مهام النظام الأخرى دون أي أولوية.

•دليل الاختناق الزمني (الاستنتاج الأهم) :نلاحظ أن المهمة دخلت الطابور (Pushed) في الساعة 20:34:01، ولكنها لم تكتمل (Completed) إلا في الساعة 20:34:23 أي أن معالجة هذا الطلب استغرقت 22 ثانية كاملة، هذا التأخير الكبير هو دليل قاطع على حدوث اختناق الطوابير (Queue Congestion) فالعامل (Worker) كان مشغولاً بمهام سابقة متراكمة، مما أجبر هذا الطلب على الانتظار طويلاً قبل أن يحصل على دوره في التنفيذ.

لذلك انتقلنا إلى بنية المشرفين المتعددين (Multi-Supervisor) ، عبر فصل المهام واستحداث مشرف مستقل ومخصص للعمليات الحساسة باسم (supervisor-payments)

```

'defaults' => [
  'supervisor-default' => [
    'connection' => 'redis',
    'queue' => ['default'],
    'balance' => 'auto',
    'autoScalingStrategy' => 'time',
    'maxProcesses' => 10,
    'memory' => 128,
    'tries' => 1,
    'timeout' => 60,
  ],
  'supervisor-payments' => [
    'connection' => 'redis',
    'queue' => ['payments'],
    'balance' => 'simple',
    'processes' => 5,
    'maxProcesses' => 5,
    'memory' => 128,
    'tries' => 3,
    'timeout' => 90,
  ],
],

'environments' => [
  'production' => [
    'supervisor-default' => [
      'maxProcesses' => 20,
    ],
    'supervisor-payments' => [
      'processes' => 5,
      'maxProcesses' => 5,
    ],
  ],
  'local' => [
    'supervisor-default' => [
      'maxProcesses' => 3,
    ],
    'supervisor-payments' => [
      'processes' => 3,
      'maxProcesses' => 3,
    ],
  ],
],

```

أسباب التغيير :

- عزل الموارد الحسابية: (Resource Isolation) منع جوبات الدفع الثقيلة من الانتظار خلف طلبات النظام العادية في طابور واحد (مما حلّ مشكلة تأخير الـ 22 ثانية للطلب 809).
- تخفيف الاختناق: حماية جداول قاعدة البيانات من الأقفال المتلاحقة (Database Locks) عبر فصل مسارات المعالجة.

الفوائد الهندسية المحققة :

- استراتيجية موازنة ذكية: الطابور العادي يتوسع ديناميكياً حسب الحاجة ($\text{balance} \Rightarrow \text{auto}$).
- طابور المدفوعات محجوز له 3 عمال مستقرين ومستعدين دائماً محلياً ($\text{processes} = 3$) لمعالجة الدفع فوراً دون أي تأخير ميكروي.

- رفع متانة النظام عند الفشل (Fault Tolerance)
 - الجوبات العادية تحاول مرة واحدة فقط ($tries \Rightarrow 1$) لمنع هدر الموارد.
 - جوبات الدفع الحساسة مُنحت 3 محاولات ($tries \Rightarrow 3$) ومهلة أكبر 90 ثانية ($timeout \Rightarrow 90$) لضمان عدم ضياع عملية الدفع في حال حدوث مشاكل في شبكة البوابة الخارجية.
- التحكم بالطاقة الاستيعابية حسب البيئة (Capacity Control):
 - في الـ local حصر العمليات بـ 3 عمال لتناسب موارد الجهاز المحلي.
 - في الـ production رفع السقف تلقائياً إلى 20 عاملاً للـ default و 5 عمال للـ payments لمنع أي طفرة طلبات متزامنة في السيرفر الحقيقي.

Main										
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+ Command
508	nour	20	0	6072	5300	3616	S	0.0	0.1	0:00.02 -bash
577	mysql	20	0	2823M	466M	38692	S	0.0	12.3	0:00.93 /usr/sbin/mysqld
1004	root	20	0	3188	1124	980	S	0.0	0.0	0:00.00 /init
1005	root	20	0	3188	1128	980	S	0.0	0.0	0:00.25 /init
1012	nour	20	0	6068	5384	3644	S	0.0	0.1	0:00.27 -bash
1742	root	20	0	3188	1124	980	S	0.0	0.0	0:00.00 /init
1744	root	20	0	3188	1256	1108	S	0.0	0.0	0:01.65 /init
1747	nour	20	0	6196	5424	3648	S	0.0	0.1	0:00.59 -bash
9100	mysql	20	0	2823M	466M	38692	S	0.0	12.3	0:02.93 /usr/sbin/mysqld
9102	mysql	20	0	2823M	466M	38692	S	0.0	12.3	0:02.25 /usr/sbin/mysqld
9709	nour	20	0	122M	58696	23480	S	0.0	1.5	0:04.27 /usr/bin/php8.3 artisan horizon:supervisor desktop-rlm643g-Hq3r:supervisor-default
9710	nour	20	0	122M	58780	23528	S	0.7	1.5	0:04.35 /usr/bin/php8.3 artisan horizon:supervisor desktop-rlm643g-Hq3r:supervisor-paymen
9727	nour	20	0	122M	59944	23856	S	0.0	1.5	0:04.63 /usr/bin/php8.3 artisan horizon:work redis --name=default --supervisor=desktop-rl
11001	root	20	0	3188	1124	980	S	0.0	0.0	0:00.00 /init
11003	root	20	0	3188	1256	1108	S	0.0	0.0	0:00.16 /init
11006	nour	20	0	6068	5384	3644	S	0.0	0.1	0:00.24 -bash
11048	mysql	20	0	2823M	466M	38692	S	0.0	12.3	0:01.04 /usr/sbin/mysqld

- المشرف العام لقنوات النظام.(supervisor-default)
- المشرف المخصص لقنوات الدفع والعمليات الثقيلة.(supervisor-payments)

- انخفاض زمن المعالجة المتراكم وصدمة الاستجابة: (TIME+) رُصد هبوط حاد وسريع في مؤشر وقت المعالجة الحسابي الكلي للـ Workers ليتراجع من 17 ثانية ليصل إلى 4 ثوان فقط. (0:04.35)

نرى ثبات مؤشر استهلاك المعالج اللحظي (CPU%) عند قيم صفيرية في الحالتين لأن جوبات المعالجة تعتمد أساساً على كفاءة الإدخال والإخراج (I/O-Bound) والانتظار الشبكي لبوابات الدفع وقاعدة البيانات، وليس على العمليات الحسابية

المعقدة (CPU-Bound). ولذلك، فإن المقياس الهندسي الأدق المعبر عن تحسن الأداء في الفحص هو تراجع الوقت المتراكم لتنفيذ العمليات (TIME+) من 17 ثانية إلى 4 ثوانٍ نتيجة عزل الطوابير

و بالرجوع لل horizon نلاحظ بالصورة التالية:

Completed Jobs			
Job	Queued	Completed	Runtime
ProcessOrder Queue: payments Tags: App\Models\Order:759	2026-05-18 19:18:41	2026-05-18 19:18:49	1.28s
ProcessOrder Queue: payments Tags: App\Models\Order:757	2026-05-18 19:18:41	2026-05-18 19:18:48	1.47s
ProcessOrder Queue: payments Tags: App\Models\Order:758	2026-05-18 19:18:41	2026-05-18 19:18:48	1.25s
ProcessOrder Queue: payments Tags: App\Models\Order:756	2026-05-18 19:18:41	2026-05-18 19:18:48	1.21s
ProcessOrder Queue: payments Tags: App\Models\Order:754	2026-05-18 19:18:41	2026-05-18 19:18:47	1.20s
ProcessOrder Queue: payments Tags: App\Models\Order:755	2026-05-18 19:18:41	2026-05-18 19:18:47	1.17 

يُلاحظ من توقيت الجوبات المتتالية لـ `ProcessOrder` أنها دفعت كلها في نفس الثانية.(19 : 18 : 41)

المعالجة الفعلية لكل جوب (Runtime) استغرقت ثانية وبضع أجزاء فقط تتراوح بين 1.17 s إلى 1.47 s واكتملت فوراً. هذا يثبت التخلص التام من اختناق الـ 22 ثانية السابق، حيث يعالج العمال المهام بالتوازي اللحظي بمجرد دخولها الطابور

Overview			
Jobs Per Minute	Jobs Past Hour	Failed Jobs Past 7 Days	Status
0	61	2	Inactive
Total Processes	Max Wait Time	Max Runtime	Max Throughput
4	-	payments	payments
Current Workload			
Queue	Jobs	Processes	Wait
default	0	1	A few seconds
payments	0	3	A few seconds

إجمالي العمليات المستقرة: تظهر اللوحة حجز 4 عمليات متوازية (Total Processes: 4) بشكل نشط ومستمر في الخلفية.

توزيع العمال الذكي: تم تخصيص عامل واحد (Processes: 1) لطابور المهام العادية default، بينما تم حجز 3 عمال متفرغين (Processes: 3) لطابور المدفوعات payments. هذا يتضمن تصفير زمن الانتظار (Wait: A few seconds) وجعل الموارد متأهبة دائماً لاستقبال أي ضغط متزامن.

Metrics	
Jobs	Queues
Queue	
default	
payments	

يؤكد تبويب القياسات نجاح عزل النظام برمجياً إلى قناتين مستقلتين تماماً هما default و payments مما يتيح تتبع ومراقبة مؤشرات أداء كل قطاع (Business Domain) في النظام بشكل منفصل ودقيق لضمان جودة الخدمة .

3- المتطلب الثالث : التحقق من المعالجة غير المتزامنة وأنظمة الطوابير (Asynchronous Queues):

قمنا بتطبيق هذا المتطلب أثناء توليد الفاتورة الخاصة بالمستخدم عند عملية الطلب.

تم الانتقال بالنظام من النمط المتزامن التقليدي (Synchronous Blocking) إلى النمط غير المتزامن (Asynchronous Non-blocking) باستخدام أنظمة الطوابير المتقدمة. أثبتت نتائج الاختبار العملية باستخدام K6 و Laravel Horizon تقليل زمن استجابة واجهة برمجة التطبيقات (API Response Time) من عدة ثوان إلى 89 ميلي ثانية مع ضمان معالجة المهام الثقيلة في الخلفية دون التأثير على تجربة المستخدم.

- المعمارية :

- تعتمد المعمارية الحديثة للنظام على فصل المهام (Decoupling) عبر تجزئة عملية الشراء إلى مرحلتين أساسيتين :
- ١- المسار الحرج : جلب الطلب , والتحقق من حالته , وإطلاق مهمة الدفع الخلفية ثم إعادة استجابة فورية للمستخدم برقم الحالة 202.
 - ٢- المسار الغير متزامن: يلتقط خادم الطوابير المهمة و يتصل ببوابة الدفع Stripe و عند النجاح , يطلق مهمة فرعية مستقلة تماماً لتوليد الفاتورة بصيغة PDF وحفظها .

- منهجية و جاهزية الاختبار :

لإثبات نجاح الفصل المعماري تم اعتماد استراتيجيات Heavy Load Simulation وفق الخطوات التالية :

```

class GenerateInvoicePDF implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    protected $order;

    public function __construct(Order $order)
    {
        $this->order = $order;
    }

    public function handle()
    {
        logger()->info("[PDF Job] Started generating invoice for Order #{$this->order->id}");

        sleep(5);

        $order = $this->order->load(['orderItems.product', 'user']);

        $data = [
            'title' => ' invoice number ' . $this->order->id,
            'id' => $order->id,
            'date' => $order->created_at->format('d/m/Y'),
            'name' => $order->user->name,
            'items' => $order->orderItems,
            'total' => $order->total_price,
        ];

        $pdf = Pdf::loadView('invoices.order_pdf', $data);
        $path = 'invoices/invoice_' . $order->id . '.pdf';
        if (!Storage::exists('public/invoices')) {
            Storage::makeDirectory('public/invoices');
        }

        Storage::disk('public')->put('invoices/invoice_' . $this->order->id . '.pdf', $pdf->output());
        $order->invoice_path = $path;
        $order->save();

        $this->order->update([
            'processed_by' => $this->queue
        ]);
        logger()->info("PDF Generated for Order #{$order->id}");
    }
}

```

- تفعيل الدالة sleep(5) ضمن الوظيفة الخاصة بتوليد الفاتورة (GenerateInvoicePDF) و اجبارها على استهلاك 5 ثوان كاملة تحاكي برمجيا معالجة ملفات PDF ضخمة ومعقدة تحت البيئة التي نحاكها.
- تم تصميم script خاص باستخدام K6 لضخ طلبات متوازية و متزامنة في نفس الجزء من الثانية عبر مستخدمين وهميين (VUs).

اختبار استجابة واجهة التطبيقات الأمامية (Client-Side Performance Test)
 تم تشغيل اختبار k6 لإرسال طلبات دفع متوازية لعدة طلبات معلقة في قاعدة البيانات للتأكد من سلوك الـ API.

```

  V K C

execution: local
script: async_test.js
output: -

scenarios: (100.00%) 1 scenario, 3 max VUs, 10m30s max duration (incl. graceful stop):
* default: 3 iterations shared among 3 VUs (maxDuration: 10m0s, gracefulStop: 30s)

INFO[0000] [k6] API Response for Order 1004: 202 | Time: 89ms source=console
INFO[0000] [k6] API Response for Order 1002: 202 | Time: 89ms source=console
INFO[0000] [k6] API Response for Order 1003: 202 | Time: 89ms source=console

■ TOTAL RESULTS

checks_total.....: 6 66.257919/s
checks_succeeded...: 100.00% 6 out of 6
checks_failed.....: 0.00% 0 out of 6

✓ is status 202 (Accepted)
✓ response is fast (< 500ms)

HTTP
http_req_duration.....: avg=89.2ms min=88.98ms med=89.3ms max=89.3ms p(90)=89.3ms p(95)=89.3ms
{ expected_response:true }...: avg=89.2ms min=88.98ms med=89.3ms max=89.3ms p(90)=89.3ms p(95)=89.3ms
http_req_failed.....: 0.00% 0 out of 3
http_reqs.....: 3 33.12896/s

EXECUTION
iteration_duration.....: avg=90.28ms min=90.22ms med=90.28ms max=90.34ms p(90)=90.33ms p(95)=90.33ms
iterations.....: 3 33.12896/s

NETWORK
data_received.....: 972 B 11 kB/s
data_sent.....: 1.6 kB 18 kB/s
```

يظهر الفحص المخبري نجاحاً مطلقاً للفحوصات بنسبة 100% (checks_succeeded: 6 out of 6). استقبل النظام الطلبات المتوازية وأعاد الرمز البروتوكولي 202 Accepted في زمن استجابة خارق قدره 89.2ms. هذا يثبت أن العميل (Client) قد تم تحريره فوراً ولم يتم إجباره على انتظار العمليات الخلفية.

تكشف سجلات Horizon الآلية الميكانيكية الحقيقية للنظام؛ حيث تظهر الوظيفة الأساسية ProcessOrder مستغرقة ثانيتين فقط لإنهاء المعاملة المادية مع Stripe ، لتقوم فوراً بوضع وظيفة الفاتورة GenerateInvoicePDF قيد التشغيل المباشر. تظهر عبارة 5s DONE متكررة بوضوح أمام وظائف الفواتير، مما يؤكد هندسياً أن الثقل الزمني البالغ 5 ثوانٍ تم عزله واستهلاكه بالكامل داخل خادم الخلفية دون أن يتأثر زمن استجابة المستخدم المقدر بـ 89ms.

```
naya@naya:~/projects/E-commerce$ php artisan horizon
```

```
INFO Horizon started successfully.
```

```
2026-05-18 14:05:02 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:05:02 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:05:04 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 14:05:04 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:05:05 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 14:05:05 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:05:05 App\Jobs\GenerateInvoicePDF ..... 906.20ms DONE
2026-05-18 14:05:10 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 14:06:50 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:06:52 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:06:53 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 14:06:53 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:06:54 App\Jobs\ProcessOrder ..... 1s DONE
2026-05-18 14:06:56 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:06:58 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 14:07:02 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 14:08:05 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:08:06 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:08:07 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 14:08:07 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:08:09 App\Jobs\ProcessOrder ..... 3s DONE
2026-05-18 14:08:09 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:08:09 App\Jobs\GenerateInvoicePDF ..... 1s DONE
2026-05-18 14:08:14 App\Jobs\GenerateInvoicePDF ..... 5s DONE
```

الخطوة الأخيرة هي مطابقة ال (TimeStamps) داخل ملفات سجلات النظام المباشرة ب (Laravel.log).

```

running (00m00.1s), 0/3 VUs, 3 complete and 0 interrupted iterations
default ✓ [=====] 3 VUs 00m00.1s/10m0s 3/3 shared iters
naya@Naya:~/projects/E-commerce$ cat storage/logs/laravel.log
[2026-05-18 12:26:05] local.INFO: Analytics Job started on: default
[2026-05-18 12:26:06] local.INFO: Analytics Job finished on: default
[2026-05-18 12:26:08] local.INFO: Analytics Job started on: default
[2026-05-18 12:26:08] local.INFO: Analytics Job finished on: default
[2026-05-18 12:37:10] local.INFO: Analytics Job started on: default
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1 | last id: 500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 501 | last id: 1000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1001 | last id: 1500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1501 | last id: 2000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 2001 | last id: 2500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 2501 | last id: 3000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 3001 | last id: 3500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 3501 | last id: 4000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 4001 | last id: 4500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 4501 | last id: 5000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 5001 | last id: 5500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 5501 | last id: 6000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 6001 | last id: 6500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 6501 | last id: 7000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 7001 | last id: 7500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 7501 | last id: 8000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 8001 | last id: 8500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 8501 | last id: 9000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 9001 | last id: 9500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 9501 | last id: 10000
[2026-05-18 12:37:12] local.INFO: Analytics Job finished on: default

```

4- المتطلب الرابع: معالجة البيانات الضخمة على دفعات (Batch Processing):

تم تطبيق هذا المتطلب أثناء عملية جرد المبيعات اليومية (إنشاء التقارير)

لإثبات كفاءة النظام، تم توليد عينة بيانات ضخمة (Data Seeding) تحاكي بيئة الإنتاج الحقيقية.
التقنية المستخدمة: تم استخدام `DB::table()->insert()` مع تعطيل قيود المفاتيح الأجنبية
`Schema::disableForeignKeyConstraints()` مؤقتاً لتسريع عملية حقن
السجلات في جدول `orders`.
حجم العينة (10,000): لتاريخ الأمس

- **المعمارية:** تم إنشاء فئة `DailySalesAnalyticsJob` وتطبيق واجهة `ShouldQueue` لإجبار إطار العمل على تحويلها إلى طابور المعالجة (Queue) بدلاً من التنفيذ المتزامن.
كود المعالجة الأساسي (تم تطبيق نمط الدفعات):


```

Order::chunkById(500, function ($orders) use (
    &$totalOrders,
    &$totalSales,
    &$paidOrders,
    &$canceledOrders
) {

    logger()->info("=== [Chunk Detected] === number of chunks in batch " . $orders->count() . "

    foreach ($orders as $order) {
        $totalOrders++;
        $totalSales += $order->total_price;

        if ($order->status === 'paid') {
            $paidOrders++;
        }

        if ($order->status === 'canceled') {
            $canceledOrders++;
        }
    }
});

```

- منهجية وجاهزية الاختبار :

- تم تكوين النظام لاستخدام محرك Redis عبر مكتبة (predis) ونظام Horizon لإدارة الطوابير. عند إطلاق الوظيفة، لم يتوقف النظام لانتظار النتيجة، بل تم إرسالها فوراً إلى الخلفية.

```
naya@Naya:~/projects/E-commerce$ php artisan horizon
```

```
INFO Horizon started successfully.
```

```

2026-05-18 12:26:08 App\Jobs\DailySalesAnalyticsJob ..... RUNNING
2026-05-18 12:26:08 App\Jobs\DailySalesAnalyticsJob ..... 406.33ms DONE
2026-05-18 12:26:20 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 12:26:20 Laravel\Telescope\Jobs\ProcessPendingUpdates 41.89ms DONE
2026-05-18 12:26:32 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 12:26:32 Laravel\Telescope\Jobs\ProcessPendingUpdates 16.97ms DONE
2026-05-18 12:26:44 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 12:26:44 Laravel\Telescope\Jobs\ProcessPendingUpdates 28.20ms DONE

```

```
^C
```

منع انهيار خادم PHP تحت ضغط البيانات، تم استخدام دالة (chunkById(500) تم توثيق التقسيم برمجياً من خلال طباعة معرفات البداية والنهاية لكل دفعة.

النتيجة الفنية: أثبت السجل أن النظام قام بسحب 500 سجل فقط في كل دورة، وانتقل بسلسلة من المعرف (ID) رقم 4001 إلى 4500، ثم من 4501 إلى 5000، وهكذا.

```

aya@Naya:~/projects/E-commerce$ tail -n 20 storage/logs/laravel.log
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 501 | last id: 1000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1001 | last id: 1500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1501 | last id: 2000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 2001 | last id: 2500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 2501 | last id: 3000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 3001 | last id: 3500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 3501 | last id: 4000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 4001 | last id: 4500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 4501 | last id: 5000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 5001 | last id: 5500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 5501 | last id: 6000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 6001 | last id: 6500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 6501 | last id: 7000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 7001 | last id: 7500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 7501 | last id: 8000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 8001 | last id: 8500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 8501 | last id: 9000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 9001 | last id: 9500
2026-05-18 12:37:12 local.INFO: Analytics Job finished on: default
aya@Naya:~/projects/E-commerce$

```

تم التحقق بنجاح من أن النظام قادر على أتمتة تقارير المبيعات اليومية بشكل موثوق. تعمل الوظيفة بشكل خفي (Asynchronous) ، تحمي الخادم من الانهيار (Chunking) ، وتولد تقريراً دقيقاً يُحفظ في جدول `daily_sales_reports` بكفاءة عالية. (High Performance)

4- المتطلب الخامس والأخير : (Load Balancing):

تم استخدام استراتيجية توزيع تعتمد على modulo hashing لتوجيه الطلبات إلى عدة طوابير تمثل خوادم مستقلة افتراضياً، بهدف محاكاة توزيع الحمل وتحسين استقرار النظام تحت الضغط

يحدد رقم الطلب الـ queue المستهدفة باستخدام المعادلة:

$$queue = (order_id \bmod 3) + 1$$

```

// distribute jobs between workers
$workerId = ($order->id % 3) + 1;
$targetQueue = "server_" . $workerId;

ProcessOrder::dispatch(
    $order,
    Auth::id(),
    $request->scenario
)->onQueue($targetQueue);

```


ال Load Balancing هو استقبال تدفق كبير من البيانات أو الطلبات في نفس اللحظة، وتوزيعها بذكاء وهنا استخدمت خوارزمية التناوب Round Robin على مجموعة من الموارد (الخوادم) لضمان عدم اختناق سيرفر واحد، مما يضمن بقاء النظام سريعاً ومستقراً تحت الضغط.

قمنا بتعديل horizon و إضافة supervisor_servers من أجل محاكاة عمل 3 سيرفرات وتوزيع الحمل عليها :

```
'supervisor-server-1' => [
  'connection' => 'redis',
  'queue' => ['server_1'],
  'balance' => 'simple',
  'processes' => 2,
  'tries' => 3,
],

'supervisor-server-2' => [
  'connection' => 'redis',
  'queue' => ['server_2'],
  'balance' => 'simple',
  'processes' => 2,
  'tries' => 3,
],

'supervisor-server-3' => [
  'connection' => 'redis',
  'queue' => ['server_3'],
  'balance' => 'simple',
  'processes' => 2,
  'tries' => 3,
],
],
```

هذه الصور أيضا تظهر النتائج :

```
naya@Naya: ~/projects/E-cor × + v
naya@Naya:/mnt/c/WINDOWS/system32$ cd ~/projects/E-commerce
naya@Naya:~/projects/E-commerce$ tail -n 20 storage/logs/laravel.log
[2026-05-18 14:34:36] local.INFO: [PDF Job] Started generating invoice for Order #1004
[2026-05-18 14:34:36] local.INFO: Order #1003: Payment processed by . Invoice sent to queue.
[2026-05-18 14:34:36] local.INFO: [PDF Job] Started generating invoice for Order #1003
[2026-05-18 14:34:37] local.INFO: PDF Generated for Order #1003
[2026-05-18 14:34:39] local.INFO: Order #1002: Payment processed by . Invoice sent to queue.
[2026-05-18 14:34:39] local.INFO: [PDF Job] Started generating invoice for Order #1002
[2026-05-18 14:34:40] local.INFO: PDF Generated for Order #1004
[2026-05-18 14:34:44] local.INFO: PDF Generated for Order #1002
[2026-05-18 19:14:59] local.INFO: Background Processing: Order #3 is being handled by Worker: server_1
[2026-05-18 19:14:59] local.INFO: Background Processing: Order #2 is being handled by Worker: server_3
[2026-05-18 19:14:59] local.INFO: Background Processing: Order #1 is being handled by Worker: server_2
[2026-05-18 19:15:01] local.INFO: Order #2: Payment processed by server_3. Invoice sent to queue.
[2026-05-18 19:15:01] local.INFO: Order #3: Payment processed by server_1. Invoice sent to queue.
[2026-05-18 19:15:02] local.INFO: Order #1: Payment processed by server_2. Invoice sent to queue.
[2026-05-18 19:15:02] local.INFO: [PDF Job] Started generating invoice for Order #2
[2026-05-18 19:15:02] local.INFO: [PDF Job] Started generating invoice for Order #3
[2026-05-18 19:15:05] local.INFO: [PDF Job] Started generating invoice for Order #1
[2026-05-18 19:15:07] local.INFO: PDF Generated for Order #3
[2026-05-18 19:15:07] local.INFO: PDF Generated for Order #2
[2026-05-18 19:15:10] local.INFO: PDF Generated for Order #1
naya@Naya:~/projects/E-commerce$ |
```

هذه الصورة توضح بدقة تشغيل الثلاث سيرفرات في نفس الوقت مما يدل على ان الحمل فعلا يتوزع عليها .

هذا يثبت مبدأ فصل الاهتمامات (Separation of Concerns) بمجرد اكتمال الدفع، لم يقم النظام بتوليد الفاتورة فوراً ليجعل العميل ينتظر، بل أرسل "أمر توليد الفاتورة" إلى طابور جديد وانتهت مهمة الدفع. هذا يضمن سرعة الاستجابة للمستخدم (Response Time) بأجزاء من الثانية.

عملية إنشاء ملفات الـ PDF معقدة وتستهلك موارد المعالج (CPU) السجلات توضح أن معالجة فاتورة واحدة استغرقت حوالي 5 ثوانٍ. لو كانت هذه العملية متزامنة (Synchronous) ، لتجمدت شاشة المستخدم لمدة 5 ثوانٍ لكل عملية شراء. لكن نقلها كـ Background Job (تنفيذاً للبند الرابع) جعلها تُعالج بهدوء في الخلفية دون أي تأثير على أداء خادم الويب الأساسي (Nginx) أو واجهة المستخدم.

```
naya@Naya:~/projects/E-commerce$ sudo service redis-server start
[sudo: authenticate] Password:
naya@Naya:~/projects/E-commerce$ php artisan horizon

INFO Horizon started successfully.

2026-05-18 19:14:59 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 19:14:59 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 19:14:59 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 19:15:01 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 19:15:01 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 19:15:02 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 19:15:02 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 19:15:02 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 19:15:05 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 19:15:07 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 19:15:07 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 19:15:10 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 16.13ms DONE
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 7.26ms DONE
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 7.24ms DONE
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 15.36ms DONE
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 7.47ms DONE
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 7.09ms DONE
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 8.41ms DONE
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 9.19ms DONE
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 10.16ms DONE
```

هنا ال loggers تظهر أيضا اثبات توزيع الحمل بالتوازي في نفس الثانية (19:14:59) ، دخلت 3 وظائف دفع (Payment Jobs) مختلفة إلى النظام. لو كان النظام يعتمد على خادم معالجة واحد (Single Worker) ، لتم تنفيذها بالتوالي (واحدة تلو الأخرى) ، مما يسبب تأخيراً للمستخدمين. ولكن بسبب توزيع الأحمال الذي قمنا ببنائه عبر خوارزمية التناوب وتحجيم المشرفين (supervisor-server-1, 2, 3) في خادم Horizon ، تم التقاط الوظائف الثلاث في نفس الأجزاء من الثانية ومعالجتها بالتوازي (Parallel Execution) ، وهذا هو صلب مفهوم الـ High Availability وتوزيع الأحمال."

ملاحظة:

- ١- أثناء العمل وجدنا أنه الطلبات متقاطعة مع بعضها بشكل كبير ما أدى إلى تقاطع في المعالجة وتعقيد في الفصل بين الحالات.
- ٢- هذا التقرير قام أكثر من عضو في الغروب التعديل عليه لذلك من الممكن أن يكون هناك اختلاف في طريقة الكتابة.